Otimização de CI/CD para Aplicações NestJS

22/03/2026

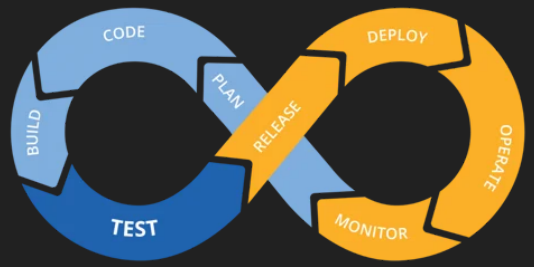
Sumário

- Contexto
  - O que é CI/CD?
  - Estrutura do Projeto
  - Tecnologias e Arquitetura Utilizadas
- O Problema
- Melhorias Implementadas
  - Padronização de Mocks
  - Testes Unitários sem Dependências Externas
  - Testes E2E com Shard e Parallel
- Resultados Obtidos
- Conclusão

Contexto

O que é CI/CD?

- **Continuous Integration (CI)**: Refere-se a integração continua do código-fonte da aplicação no repositório[1] principal.
- **Continuous Deployment (CD)**: Extensão do processo acima, garantindo que as mudanças sejam automaticamente implantadas nos ambientes[2] referentes (produção/testes)



[1] *Um repositório é como uma **pasta compartilhada com histórico**.*
[2] *Um ambiente resumidamente se refere ao **lugar onde o código final roda**.*

Benefícios do CI/CD

- **Maior agilidade:** O feedback ágil do processo de implantação reduz o tempo entre a escrita e disponibilização do código.
- **Maior segurança e frequência:** O processo automatizado evita o desgaste comum de processos manuais de deploy/validação.
- **Antecipação de Erros:** Testes automatizados servem como um bloqueio final para problemas não capturados em testes locais.
- **Menos retrabalho:** Erros antecipados resultam em correções antecipadas, evitando tropeços no meio do caminho.

Estrutura do Projeto

O projeto conta com quatro ambientes distintos:

- **Desenvolvimento**: Ambiente utilizado pelos desenvolvedores para testes e implementação de novas funcionalidades.
- **Homologação**: Utilizado pelos QAs (Quality Assurance) para validação de novas funcionalidades antes da liberação.
- **Demonstração**: Um ambiente semelhante à produção, usado como Sandbox por nossos parceiros.
- **Produção**: O ambiente final onde a aplicação é utilizada pelos usuários.

Estrutura do Projeto

Nosso pipeline de CI/CD é dividido em duas partes principais:

1. **CI de MRs/PRs**: Responsável por validar a integridade do código antes de ser mesclado na branch do ambiente.
2. **CI/CD da branch do ambiente**: Valida novamente o código e realiza a implantação no ambiente correspondente.

HotFixes são aplicados diretamente em demonstração, validados e depois propagados para os outros ambientes.

MR (Merge Request) é um processo utilizado em plataformas como GitLab para solicitar a integração de um conjunto de alterações em uma branch principal. É similar ao PR (Pull Request) utilizado no GitHub.

Tecnologias e Arquitetura Utilizadas

O projeto segue uma arquitetura de **microsserviços**, onde cada serviço é responsável por uma funcionalidade específica e se comunica com outros serviços por meio de APIs e mensageria.

A otimização realizada neste contexto foi aplicada a um dos **microsserviços críticos da operação**, que desempenha um papel essencial no funcionamento do sistema. As tecnologias principais utilizadas incluem:

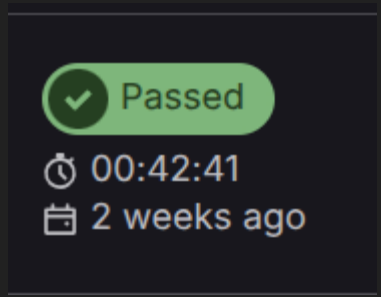
- **NestJS** para a aplicação backend.
- **GitLab** como ferramenta de CI/CD e versionamento de código.
- **Jest** para testes unitários e de integração.
- **Docker** para conteinerização e gerenciamento de dependências.

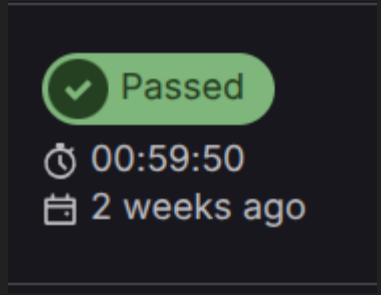
O Problema

O Problema

Um intervalo demasiadamente elevado para uma que uma correção crítica alcançasse produção, tomando em conta:

- **~40 minutos** para a MR de demonstração.
- **~1 hora** para a branch de demonstração.
- **~40 minutos** para a MR de produção.
- **~1 hora** para a branch de produção.





Esse tempo adicional em relação a MR se dá por conta de etapas exclusivas ao processo de implantação.

Melhorias Implementadas

Padronização de Mocks

Problema

Diferentes abordagens para os mocks de teste resultavam em inconsistências e dificuldades na manutenção. Além disso, essa falta de padrão causava erros intermitentes no processo de CI/CD, quebrando os testes automatizados. Nisso, era necessário **rodar a mesma job inúmeras vezes até obter uma execução bem sucedida**, o que adicionava ao tempo de execução da pipeline.

Solução

Padronização de mocks por meio da biblioteca **Rosie**, tornando os testes mais consistentes e sustentáveis em manutenção.

```
export const agentConfigFactory = Factory.define<AgentConfigEntity>(
  'AgentConfigEntity',
)
  .attrs({
    id: () => faker.string.uuid(),
    //...other props
  })
  .after(toInstance(AgentConfigEntity));
```

Testes Unitários sem Dependências Externas

Problema

Havia dependência direta de muitos serviços "reais" (Redis, Postgres, Localstack) que não cabiam ao escopo de um teste unitário[1], causando um overhead desnecessário na execução.

Solução

Remoção de todas dependências em favor de mocks, reduzindo significativamente o tempo de execução.

[1] *Teste isolado de uma unidade de código, focando em sua implementação sem nenhuma dependência externa.*

Testes Unitários sem Dependências Externas

```yaml
services:
  app:
    ports: []
    stdin_open: true
    tty: true
    image: ${CONTAINER_RELEASE_IMAGE-payments:ci}
    build:
      context: .
      target: ci
      dockerfile: Dockerfile
    env_file:
      - .env.test
    depends_on:
      - db-test
      - redis
      - localstack
    networks:
      - ci
```

Testes Unitários sem Dependências Externas

Antes:

```
specsUnit:
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker pull $CONTAINER_RELEASE_IMAGE
    - docker-compose -f docker-compose.ci.yml run --rm app yarn
    - docker-compose -f docker-compose.ci.yml run --rm app yarn db:create
    - docker-compose -f docker-compose.ci.yml run --rm app yarn db:migrate
    - docker-compose -f docker-compose.ci.yml run -e ci=true --rm app yarn test:u
```

Depois:

```
specsUnit:
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker pull $CONTAINER_RELEASE_IMAGE
    - docker-compose -f docker-compose.ci.yml run -e ci=true --rm --no-deps app yarn test:u
```

Testes E2E com Shard e Parallel

O que é Shard no Jest?

Shard se refere a fragmento em inglês, que exemplifica o processo de Sharding no Jest, que implica a execução dos testes em "fragmentos" menores do processo total.

Exemplo de uso do Shard no Jest:

```
jest --shard=1/3
```

No exemplo acima, estamos executando o primeiro de três shards, ou seja, dividimos os testes em três partes e estamos rodando apenas uma delas.

O que é Parallel no GitLab CI?

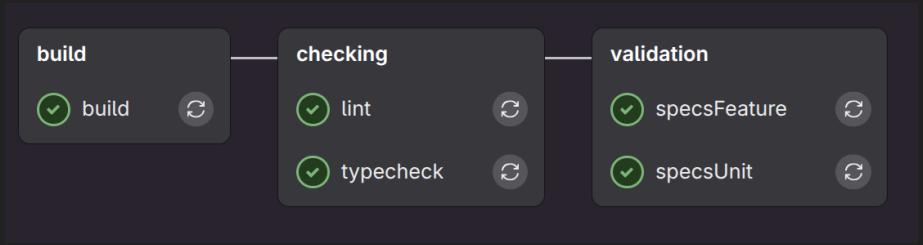
Parallel no GitLab CI permite rodar múltiplas instâncias de uma mesma job simultaneamente, distribuindo a execução dos testes em diferentes máquinas virtuais.

Exemplo de Parallel no GitLab CI:

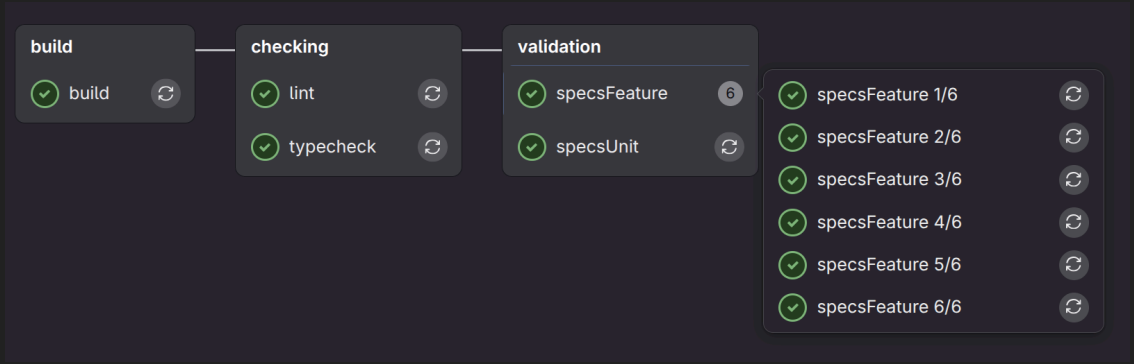
```
test-e2e:
  script:
    - npm run test:e2e
  parallel: 3 # Executará a job em 3 instâncias simultâneas
```

O que é Parallel no GitLab CI?

Antes:



Depois:

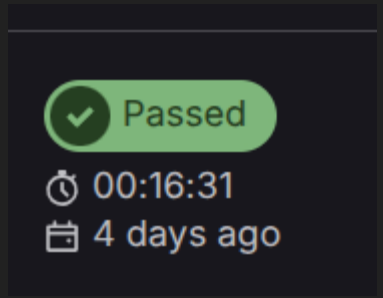


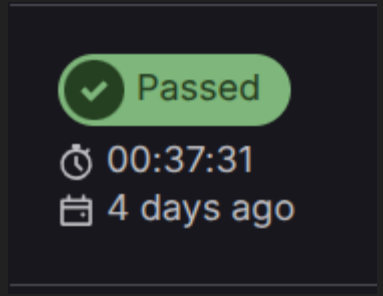
Resultados Obtidos

Resultados Obtidos

Após a implementação das otimizações, os tempos de execução do CI/CD foram significativamente reduzidos:

- **CI de MRs/PRs** passou de ~40 minutos para **15 minutos**.
- **CI/CD da branch do ambiente** passou de ~1 hora para **35 minutos**.





Conclusão

Referências

- Jest Shard
- Speed up your Jest tests with shards

Slides · GitHub

Powered by Slidev

Muito obrigado!

Se ficou com dúvidas a respeito de uma ou mais partes deste documento, não hesite em entrar em contato conosco.

zrp.com.br
luciano.weslen@zrp.com.br